



Python Prerequisites

- Data Types
- Data Structures
- Copies and Pointers in Python
- Object Oriented Programming Principles

Data Types

- **Numbers:**
 - Integers, declared as `int`
 - Decimals, declared as `float` (from *floating point*)
- **Strings:**
 - characters sequences declared between single or double apexes. Is also possible the positional access:
 - ```
s = "Hi, I am Matteo! :)"
print(s[2]) # Output: ", "
```
- **Booleans:**
  - True o False, declared as `bool`
  - The function **`type()`** returns the data type of the value between its brackets:
    - ```
type(1) # Output 'int'
print(type(1.0)) # Output: 'float'
```

Data Types: Some Details

- Functions **int()** and **float()** can be used to *convert data types*:

- `x = 2.5`
`y = int(x)`
`print(y)` # Output: "2"
- **Automatic conversion** with numbers performed by Python when possible:

- `x = 5`
`y = 2`
`z = x / y`
`print(z)` # Output: "2.5"

- Pay attention to *not mix single and double apexes*:

- `"I am 'Matteo'"` returns an **error**

- The function **str()** can be used to convert a number in string:

- `x = 4`
`type(x)` # Output: 'int'
`str(x)`
`type(x)` # Output: 'string'

- **Escape characters**:

- `\n` for a new line
- `\t` to insert a tab (usually defined as 4 consecutive spaces)
- `\` escape command for special characters in a string (e.g., `"", "'", '\')`

Operators: Assignment

- (re)assign a value to a variable:
 - ❖ **Assignment (=)**: assigns to the variable on the left side of the “equal” (“=”) the value on the right side of the “equal” (“=”, e.g., $x = y*2$)
 - ❖ **Assignment with sum (+=)**: adds to the current value of the variable on the left side of the “equal” (“=”), the value on the right side. (e.g., $x = 2$, then $x += 3$ causes $x == 5$)
 - ❖ **Assignment with subtraction (-=)**: deducible from above
 - ❖ **Assignment with multiplication (*=)**: deducible from above
 - ❖ **Assignment with division (/=)**: deducible from above
 - ❖ **Assignment with module (%=)**: deducible from above
 - ❖ **Assignment with power (**=)**: deducible from above
 - ❖ **Assignment with integer division (//=)**: deducible from above

Operators: Arithmetic

- Usually applicable to numbers, the result is a number:
 - **Sum (+)**: applicable also on strings!
 - **Subtraction (-)**
 - **Multiplication (*)**: applicable also on strings!
 - **Division (/)**: result is a float, if *necessary* (e.g., $5 / 2 = 2.5$)
 - **Power (**)**: e.g., $3 ** 2 = 9$
 - **Integer division (//)**: e.g., $5 // 2 = 2$
 - **Module (%)**: remainder of the integer division (e.g., $5 \% 2 = 1$)

Operators: Comparison

- *Applicable to any type of data, the result is a Boolean (i.e., **True** o **False**):*
 - **Equality** (==): determines whether two values are equal (i.e., same type, same value)
 - **Inequality** (!=): the opposite
 - **Greater than** (>): also works on strings, considering alphanumeric order (uppercase before lowercase)
 - **Less than** (<): as above, but in reverse
 - **Greater than or equal to** (>=): greater inclusive
 - **Less than or equal to** (<=): inclusive

Logical Operators

- *Applicable to Booleans, the result is a Boolean*
 - **AND:** the result is **True** if and only if ***both operands are True***
 - **OR:** the result is **True** provided that ***at least one operand is True***
 - **NOT:** the result is the ***negation of the operand***

X	Y	X && Y
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
TRUE	TRUE	TRUE
FALSE	FALSE	FALSE

X	Y	X Y
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
TRUE	TRUE	TRUE
FALSE	FALSE	FALSE

On the left, the **AND case table of truth**; on the right, the **OR**.

Operators: if ... else, if ... elif ... else

- Statement to execute code *if and only if* the condition in parentheses is true, *otherwise* other code is executed.
 - Structure:

```
if (<condition returning true or false>):  
    <instructions>
```
 - E.g.:

```
if ((x % 2) == 0):  
    s = "x is even"  
else:  
    s = "x is uneven"  
print(s)
```
- **Indentation in Python is not aesthetic, but fundamental**
- The condition can be arbitrarily complex (e.g., conjunction of conditions)
- Statement to *concatenate multiple if ... else ...*
 - Structure:

```
if (<condition that returns true or false >):  
    <instructions>  
elif (<further condition that returns true or false >):  
    < further instructions>  
else:  
    <additional further instructions>
```
 - E.g.:

```
if (x % 2) == 0:  
    s = "x is even"  
elif (x % 2) != 0:  
    s = "x is uneven"  
else:  
    s = "Impossible !"  
print(s)
```

Operators: Nesting

- Multiple *if ... (else/elif) statements* can be *arbitrarily nested*.
- Pay attention to the **indentation**, it must reflect the nested structure of the instructions!
- Often translated using **AND / OR**
- E.g.:

```
if <condition returning true or false>  
AND <further condition returning ... >:  
    ... instructions  
    ... other instructions
```

```
if <condizione>:  
    if <ulteriore condizione>:  
        <istruzioni>  
elif <altra condizione>:  
    <altre istruzioni>  
else:  
    if <ancora una condizione>:  
        <ancora altre istruzioni>  
elif ...:  
    ...
```

Loops: for Loop with range(), while Loop

- *For loop with range():*
 - The **range([start,] stop [, step])** function generates a *sequence of numeric values between two extremes* ([start included, optional, default 0], stop excluded) at a *chosen distance* (step, optional, default 1)
 - Used in a "**for**" it allows you to iterate a predefined number of times, keeping track of them in a variable
 - E.g.:

```
for i in range(1, 10, 2):  
    print(i)
```
- *While loop:*
 - A statement to **repeat** a set of instructions until an arbitrary **termination condition** is met.
 - Structure:

```
while <condition that returns true  
or false>:  
    <instructions>
```
 - E.g.:

```
i=0  
while (i < 10):  
    print(i)  
    i += 1
```

Loops: break and continue Instructions

- *The break statement:*
- The **break statement** *immediately stops the execution of a loop.*
- Any following instructions **are skipped**.
- Any remaining iterations **are skipped**.
- Typically, it is used *within an if that verifies the termination condition.*

- E.g.:

```
i = 0
for (c in "hello world"):
    if (i > 5):
        break

    print(c)
    i += 1
```

- *The continue statement:*
- The **continue statement** *skips the remaining instructions in the current iteration of a loop.*
- Only the remaining instructions are skipped.
- Any remaining iterations **are executed**.
- Typically, it is used *within an if that checks the skip condition.*

- E.g.:

```
i = 0
for c in "hello world":
    if (i == 4):
        i += 1
        continue

    print(c)
    i += 1
```

Loops: Nesting

- **For** and **while** loops can also be *arbitrarily nested*.
- Beware of **break** and **continue**: they apply to the *loop to which they belong (scope)*

```
go = True
while (go) :
    for i in range(10)
        print(f'I am at the cycle number {i} ')
        if i > 5:
            i = 0
            go = False
            break
        else:
            i+=1
            print(f'Into the while(d) ')
```

Console I/O: Input from Console

- The **input()** function *allows you to wait for user input from the console, storing it in a variable choice*
- Optionally (but it is recommended) you can *enter a sentence to be shown on the console to clarify the expected input*
- The **input()** function stores the *input as a string*; if another type of data is needed then *a cast (= conversion) must be performed*.

```
cmd = input()
cmd = input('Command (up, down):')
num = float(input('Number: '))
print(type(num))      # Out: float
```

Console I/O: Output on Console

- The function **print(value [, value] [, sep=' '] [, end=' '])** allows you to *show output* (one or more values) to the *user on console*.
- Each value is separated by the string specified in the *sep* field (optionally, default '\b')
- the "print" ends with the string specified in the *end* field (optionally, default '\n')

```
print('hello', 'world', sep='---', end='!?.')
```

```
# Out: hello---world!?
```

```
s = 'Matteo'
```

```
print('hello', s)
```

```
# Out: Hello Matteo
```

Comments

- The `#` symbol identifies a single-line comment, which is text that is not interpreted as code to be executed

- E.g.:

```
for i in range(10): # 0 to 9 included
    print(i)
```

- For multi-line comments, use the 3 double quotes, also called ***docstrings***

- E.g.:

```
"""
multi-line comment as docstring
"""
```

“Code tells you how; Comments tell you why.”

— *Jeff Atwood (aka Coding Horror)*

<https://realpython.com/documenting-python-code/>

You don't write code to make machines execute, but to have it read by other humans

NOTE: docstrings were not created to comment on code, but as a tool offered to developers to document their code, and automate the process of creating documentation (hated by programmers).

Modules

- Every programming language comes with a so-called **standard library**, i.e. a collection of *ready-to-use data types and functions*.
- For Python, these functions are **collected into modules**, which are collections of features such as:
 - **string manipulation**
(<https://docs.python.org/3/library/text.html>, e.g., string)
 - **Date and time manipulation**
(<https://docs.python.org/3/library/datetime.html>)
 - **mathematical functions**
(<https://docs.python.org/3/library/numeric.html>, e.g., math)
 - **access to OS functionalities**
(<https://docs.python.org/3/library/os.html>loindemo.io/)
 - **access to the filesystem**
(<https://docs.python.org/3/library/filesys.html>)

Python Data Structures

- **Lists:** between `[]` is a list;

⚡ List specs: Implementation: re-sizable array; Mutable: yes, Insertion order: yes; Allows duplicates: yes.

```
#list with mixed data types
my_list = [1, 'Hello', 3.4]
#nested list
my_list = ['mouse', [8, 4, 6], ['a']]
```

- **Tuples:** between `()` is a tuple;

🌈 Tuple specs: Implementation: records; Mutable: no, mutable items; Insertion order: yes; Allows duplicates: yes.

```
#mixed tuple
my_tuple = (1, 'Hello', 3.4)
#nested tuple
my_tuple = ('mouse', [8, 4, 6], (1, 2, 3))
```

- **Sets:** between `{}` is a set;

💡 Set specs: Implementation: hash table; Mutable: yes; Insertion order: no; Allows duplicates: no.

```
#set of mixed datatypes
my_set = {1.0, 'Hello', (1, 2, 3)}
#set from list
my_set = set([1, 2, 3, 2])
```

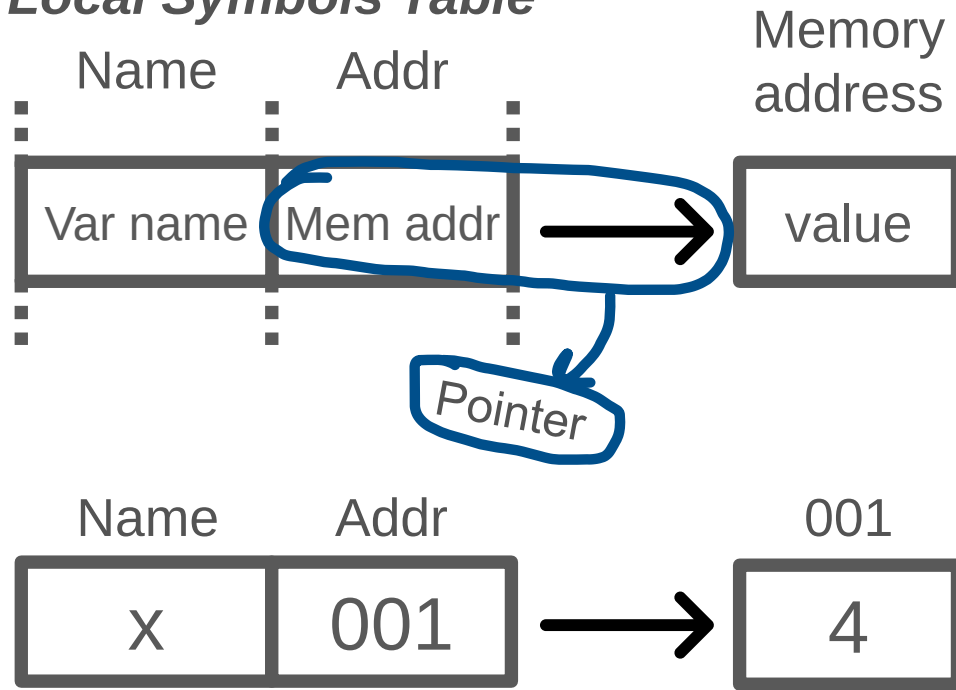
- **Dictionaries:** between `{}` with key:value pairs for each field, is a dictionary.

Implementation: hash table, Mutable: yes, Insertion order: no, Allows duplicates: no in keys, yes in values

```
#Creating a Dictionary with Integer Keys
dict = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
#Creating a Dictionary with Mixed key
dict = {'Name': 'Geeks', 1: [1, 2, 3, 4]}
```

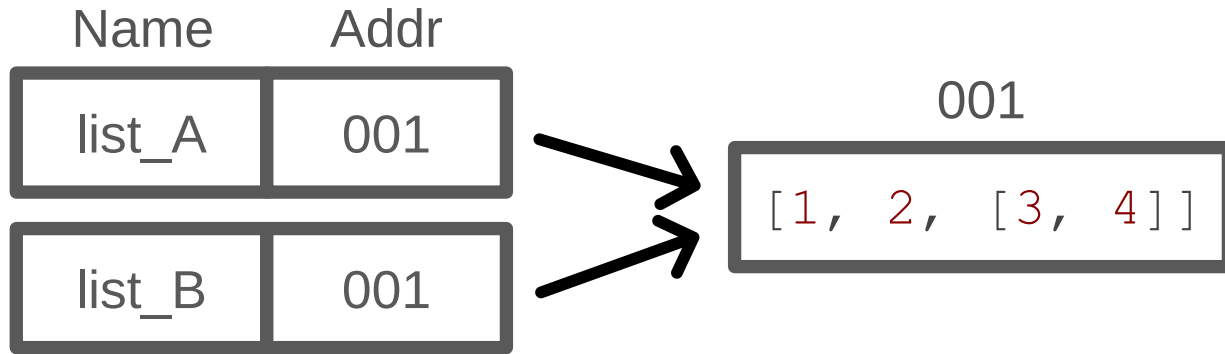
Python Copies: How Pointers Work

Local Symbols Table



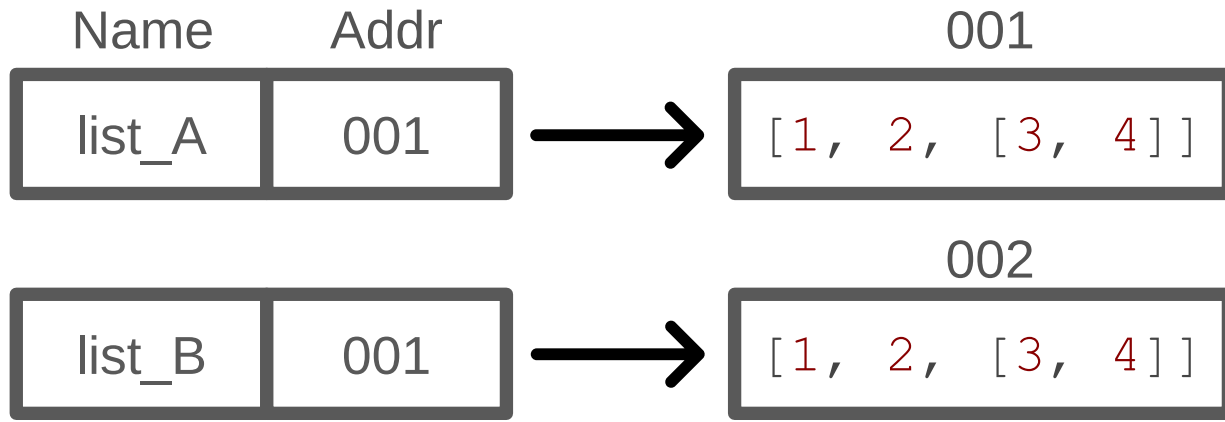
- When a number (or a general object) is assigned to a variable:
 - the variable name and the memory address where (also called pointer) the object is saved are stored in the Local Symbols Table;
 - the object is saved in a memory location at the given memory address.
- This makes copying variables a little intriguing for **mutable objects**.
- In Python *everything is an object*:
 - **Int, float, strings, tuple**, are *immutable objects*;
 - **List, dictionary, set, (personalized) objects** are *mutable objects*.

Python Copies: Shallow and Deep Copy



```
import copy
```

```
list_A = [1, 2, [3, 4]]
list_B = copy.copy(list_A)
# Modifying a nested object in the copy
list_B[2][0] = 100
print(list_A) # Out: [1, 2, [100, 4]]
print(list_B) # Out: [1, 2, [100, 4]]
print(list_A is list_B) # Out: False
print(list_A[0] is list_B[0]) # Out: True
```



```
import copy
```

```
list_A = [1, 2, [3, 4]]
list_B = copy.deepcopy(list_A)
# Modifying a nested object in the copy
list_B[2][0] = 100
print(list_A) # Out: [1, 2, [3, 4]]
print(list_B) # Out: [1, 2, [100, 4]]
print(list_A is list_B) # Out: False
print(list_A[2] is list_B[2]) # Out: False
```

Python OOP

Object-oriented programming (OOP) is a **programming paradigm** that uses **objects** and **classes** to organize and manage code.

This approach is based on key concepts such as **encapsulation**, **inheritance**, **polymorphism**, and **abstraction**. OOP is designed to facilitate the creation of modular, reusable, and maintainable software.

- **Classes:**

Classes are **models or templates** for **creating objects**. They define **attributes** (*fields*) and **behaviors** (*methods*) that are common to all objects of that type.

Example: The Car class can define attributes and methods that are common to all cars.

- **Objects:**

Objects are **instances of classes** and represent **entities** in the real world or concepts in the program. Each object has a **state** (stored in *attributes*) and a **behavior** (defined by *methods*).

Example: A Car object can have attributes like **color** and **speed**, which can be modified by methods like **accelerate()** and **brake()**.

Python OOP: Example

```
# class Car definition
class Car:
    def __init__(self, color, speed):
        self.color = color
        self.speed = speed
    def accelerate(self):
        self.speed += 10
        print(f"The car is accelerating. Actual speed: {self.speed} km/h")
    def brake(self):
        self.speed -= 10
        print(f"The car is braking. Actual speed: {self.speed} km/h")

if __name__=="__main__":
    # Creating an instance of the class Car
    my_car = Car(color="red", speed=50)
    # Utilizing methods of the object my_car
    my_car.accelerate() # Output: The car is accelerating. Actual speed: 60 km/h
    my_car.brake() # Output: The car is braking. Actual speed: 50 km/h
```